Day 06 Assignment

# حل واجب المحاضرة السادسة: ASP.NET Core MVC

حل شامل ومنسق للأسئلة المقالية والبرمجية بطريقة سهلة، إنسانية ومبسطة

Thread vs Task | Async Programming | No-Conditions View | Course CRUD & Validation | Middleware Pipeline | Kestrel Web Server | Built-in Delegates | NET 9 Generics.

## 1 مقال لينكد إن: رحلة الـ HTTP Request من خادم Kestrel إلى الـ Middleware Pipeline

💡 **فكرة المقال:** صِيغ بأسلوب احترافي مشوق يناسب شبكة LinkedIn مع تشبيه بشري رائع لفهم آلية المعالجة خلف الكواليس.

### 🚀 Ever wondered what happens when you hit Enter in ASP.NET Core? Meet Kestrel & The Middleware Pipeline!

تخيل أنك طلبت وجبة في مطعم فاخر.. مين أول شخص بيستلم طلبك من الباب؟ الجرسون! ومين بيجهز الأطباق محطة محطة في المطبخ؟ طاقم الطهاة!

ده بالظبط اللي بيحصل مع كل **HTTP Request** في عالم **ASP.NET Core**! تعالوا نلخص الرحلة في دقيقتين 👇

#### 1️⃣ البوابة الأولى: خادم Kestrel (The Gateway)

خادم **Kestrel** هو الـ Web Server الداخلي الافتراضي في .NET. خفيف وسريع جداً (Lightning-Fast) وعابر للمنصات (Cross-Platform).

- **وظيفته الأساسية:** بيستقبل البيانات الخام (Raw TCP Packets) القادمة من المتصفح عبر الشبكة، ويفكك الـ HTTP Headers والـ Body، ويحولها لكائن C# منظم اسمه `HttpContext`.
- **علاقته بالـ Reverse Proxy (IIS / Nginx):** Kestrel ممتاز وسريع جداً في معالجة الـ HTTP، بينما نضع أمامه سيرفر مثل Nginx أو IIS لمهام الـ SSL Offloading، حماية الـ Firewall، وتوزيع الحمل (Load Balancing).

#### 2️⃣ خط المعالجة: الـ Middleware Pipeline (The Assembly Line)

بمجرد أن يجهز Kestrel كائن الـ `HttpContext`، يسلمه لسلسلة محكمة من الـ **Middlewares**:

- **مبدأ الدمية الروسية (Russian Doll):** الطلب يمر من المحطة الأولى للثانية للثالثة عبر استدعاء `()next`، ولما الـ Controller يولد الـ Response، يرجع يمر بنفس المحطات بالعكس!
- **الـ Short-Circuiting (الإنهاء المبكر):** لو المستخدم غير مسجل دخول، الـ **Authentication Middleware** لن ينادي `()next`، بل سيرجع فوراً رد "401 Unauthorized" وينهي الرحلة مبكراً!
- **أشهر الأنواع واستخداماتها:**
  1. `()app.Use`: تمرير الطلب للمحطة التالية مع إمكانية تعديل الـ Request أو الـ Response.
  2. `()app.Run`: محطة نهائية (Terminal) تغلق الـ Pipeline وترجع الرد.
  3. `()app.Map`: تفرع في مسار الـ Pipeline بناءً على رابط الـ URL المطلوب.

🎯 **الخلاصة:** Kestrel يحول إشارات الشبكة إلى كائن C#، والـ Pipeline تؤمن الطلب وتوجهه للمتحكم، وتجهز الرد وترجعه للعميل بسرعة وأمان!

#DotNet #CSharp #AspNetCore #BackendDevelopment #SoftwareEngineering #WebDev

## 4 ثلاث حالات واقعية نحتاج فيها البرمجة غير المتزامنة (Asynchronous Programming)

🧠 **فكرة الـ Async ببساطة:**
الـ Async (عبر `async / await`) لا يقوم بتسريع العملية الحسابية نفسها، بل يمنع **تجميد الـ Thread** أثناء انتظار عمليات خارجية (I/O Operations) لكي يظل السيرفر قادراً على خدمة آلاف المستخدمين في نفس اللحظة!

### 1. استعلامات قواعد البيانات (Database Queries & EF Core)

عند تشغيل استعلام SQL لقراءة كميات كبيرة من البيانات أو كتابتها عبر Entity Framework Core:

```
public async Task<IActionResult> Index()
{
    // الـ Thread يتحرر فوراً لخدمة عملاء آخرين حتى ينتهي SQL Server من معالجة الاستعلام
    var courses = await _context.Courses.ToListAsync();
    return View(courses);
}
```

**السبب العملي:** سيرفر قاعدة البيانات يأخذ وقتاً (20ms - 100ms) في استرجاع البيانات من القرص الصلب. في الأسلوب المتزامن، يتجمد الـ Thread بالكامل في الانتظار؛ أما في الـ Async، فالـ Thread يعود لـ ThreadPool ليخدم مستخدماً آخر فوراً.

### 2. استدعاء الـ APIs الخارجية (External HTTP / Web Services)

عند التواصل مع خدمات وبوابات دفع إلكتروني (مثل Stripe أو فوري) أو إرسال SMS أو استدعاء خدمات الذكاء الاصطناعي:

```
public async Task<PaymentResult> ProcessPaymentAsync(PaymentDto dto)
{
    using var client = new HttpClient();
    var response = await client.PostAsJsonAsync("https://api.stripe.com/v1/charges", dto);
    return await response.Content.ReadFromJsonAsync<PaymentResult>();
}
```

**السبب العملي:** زمن استجابة خوادم الإنترنت يستغرق من 200ms إلى عدة ثوانٍ. تجميد مسارات المعالجة أثناء انتظار الرد الخارجي يؤدي سريعاً لنفاد المسارات وانهيار الخادم (Thread Starvation).

### 3. عمليات قراءة وكتابة الملفات والوسائط الكبيرة (File I/O & Streaming)

عند قيام المستخدمين برفع ملفات PDF أو صور شخصية أو قراءة ملفات تصدير Excel كبيرة:

```
public async Task SaveUploadedFileAsync(IFormFile file, string destinationPath)
{
    using var stream = new FileStream(destinationPath, FileMode.Create);
    // نقل بيانات الملف تدفقياً للقرص الصلب بدون حجز المسار البرمجي
    await file.CopyToAsync(stream);
}
```

**السبب العملي:** القراءة والكتابة على أقراص التخزين (HDD/SSD/Cloud Storage) أبطأ بآلاف المرات من المعالج والذاكرة، واستخدام الدوال غير المتزامنة يضمن استجابة وسلاسة المنظومة ككل.

## 5 الفرق الجوهري والشامل بين الـ Thread والـ Task

💡 **التشبيه البشري البسيط:**
**الـ Thread (الموظف الفعلي):** موظف ثابت تم تعيينه بمكتب وراتب ومساحة خاصة (مكلف جداً لنظام التشغيل).
**الـ Task (أمر الشغل):** ورقة أو تكليف مكتوب فيه "وصل هذا الطرد". خفيف جداً، وأي موظف متاح في مجمع الموظفين (ThreadPool) يأخذه وينفذه ثم يتفرغ لمهمة أخرى!

| وجه المقارنة | Thread (المسار المنخفض) | Task (مهمة التوازي عالية المستوى) |
|---|---|---|
| **التعريف والتجريد** | كائن تنفيذي مباشر تابع لنظام التشغيل (OS Thread). | تجريد عالي المستوى من مكتبة TPL يمثل وعداً (Promise) بعملية غير متزامنة. |
| **استهلاك الذاكرة والموارد** | ثقيل جداً (يحجز حوالي 1MB من الـ Stack لكل Thread). إنشاء 1000 مسار ينهك النظام. | خفيف جداً (بضع بايتات لكائن المهمة Task Object). يمكنك إنشاء 100,000 Task بأمان تام. |
| **إدارة الـ ThreadPool** | ينشئ Dedicated Thread خاص به ولا يعود للمسبح تلقائياً. | تدار بذكاء عبر الـ .NET ThreadPool مع إعادة تدوير المسارات المتاحة بكفاءة. |
| **إرجاع القيم (Return Value)** | صعب ومعقد (يتطلب متغيرات مشتركة واستخدام أقفال Locks). | سهل ومباشر عبر النوع العام `Task<TResult>` واسترجاع `task.Result` أو `await`. |
| **دعم async / await** | لا يدعم صيغة `await` الحديثة. | مصمم خصيصاً ليتكامل بسلاسة مع الكلمات المفتاحية `async / await`. |
| **التنسيق والسلاسل (Composition)** | معقد جداً عند محاولة تشغيل عدة مسارات بالترتيب أو انتظار اكتمالها معاً. | مرن للغاية بدوال جاهزة مثل `Task.WhenAll` و `Task.WhenAny` و `ContinueWith`. |
| **معالجة الاستثناءات والإلغاء** | أي خطأ غير معالج قد يسقط التطبيق كاملاً، ولا يدعم الإلغاء الآمن. | يجمع الاستثناءات في `AggregateException` ويدعم كائن الإلغاء القياسي `CancellationToken`. |

🎯 **متى نستخدم كل منهما؟**
في 99% من التطبيقات الحديثة نستخدم **Task** (خاصة لعمليات الـ I/O والـ Web Requests). ونلجأ لـ **Thread** فقط في حالات نادرة جداً تتطلب عملية خلفية تعمل إلى ما لا نهاية (Long-Running Background Loop) تتطلب إعدادات خاصة لأولويات المعالج على مستوى نظام التشغيل.

## 6 الميزات الجديدة في .NET 9 للتعامل مع الأنواع العامة (Generics)

أضافت مايكروسوفت في **.NET 9** إضافات نوعية للأنواع العامة لدعم الأداء وتسهيل كتابة الكود:

### 1. دوال تجميعية عامة في LINQ بدون الحاجة لـ GroupBy:

- `CountBy<TSource, TKey>` : حساب عدد تكرارات العناصر بناءً على خاصية مفتاحية بدون إنشاء كائنات وسيطة:

```
// .NET 9: كود فائق السرعة وخفيف على الذاكرة
var studentsByDept = students.CountBy(s => s.DepartmentId);
foreach (var pair in studentsByDept)
    Console.WriteLine($"Dept {pair.Key}: {pair.Value} Students");
```

- `AggregateBy<TSource, TKey, TAccumulate>` : تجميع مخصص للبيانات (مثل مجموع درجات أو رواتب) لكل مفتاح في دورة واحدة سريعة (Single-Pass).

### 2. الدالة العامة `Index<TSource>` للترقيم المباشر:

تتيح الحصول على كل عنصر مع رقم الفهرس الخاص به بدون متغير خارجي أو Indexer يدوي:

```
foreach (var (index, course) in courses.Index())
    Console.WriteLine($"{index + 1}. {course.Name}");
```

### 3. تحسينات مترجم RyuJIT في تخصيص الكود العام (Generic Specialization):

توليد كود آلي مخصص عالي السرعة للـ Generics المبنية على الـ Value Types مع إلغاء تكلفة نداءات الواجهات (Devirtualization).

## 7 أنواع الـ Built-in Delegates بالتفصيل (Action, Func, Predicate)

> 💡 **ما هو الـ Delegate؟** مؤشر آمن لنوع الدالة (Type-Safe Function Pointer). وفرت .NET 3 أنواع عامة جاهزة تغني عن إنشاء أنواع مخصصة:

### 1. الـ Action:

- **نوع العائد:** دائماً **void** (لا يرجع أي قيمة). يأخذ من 0 حتى 16 معامل.
- **استخدامه:** لتنفيذ عمليات لها تأثير جانبي (طباعة، حفظ سجلات، إطلاق حدث).

```
Action<string> logMessage = msg => Console.WriteLine($"[LOG]: {msg}");
logMessage("User logged in"); // ينفذ ولا يرجع قيمة
```

### 2. الـ Func:

- **نوع العائد:** **يرجع قيمة دائماً**. النوع الأخير في قائمة الـ Generics هو نوع القيمة المعادة! يأخذ من 0 حتى 16 معامل.
- **استخدامه:** في الحسابات والتحويلات واستعلامات الـ LINQ مثل `Select.()` .

```
Func<int, int, int> multiply = (x, y) => x * y;
int result = multiply(6, 7); // 42
```

### 3. الـ Predicate:

- **نوع العائد:** دائماً **bool**. يأخذ **معاملاً واحداً فقط** من النوع T (مكافئ لـ `Func<T, bool>` ).
- **استخدامه:** في الفحص والفلترة، مثل دوال القوائم `Find` و `Exists` .

```
Predicate<int> isSuccess = grade => grade >= 50;
bool passed = isSuccess(85); // true
```

## 2 حل وشرح السؤال الثاني: Course CRUD مع Server-Side & Client-Side Validation

✅ **حالة الكود:** تم تطبيق الـ CRUD بالكامل في مشروع `MVCGiza02Solution` مع التحقق من صحة البيانات (Server & Client Validation).

### 1. نموذج الكورس مع قواعد التحقق (Course Model with Data Annotations):

```
public class Course
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Course Name is required")]
    [MinLength(2, ErrorMessage = "Course Name must be at least 2 characters")]
    [MaxLength(50, ErrorMessage = "Course Name cannot exceed 50 characters")]
    [Display(Name = "Course Name")]
    public string Name { get; set; } = string.Empty;

    [Required(ErrorMessage = "Total degree is required")]
    [Range(50, 100, ErrorMessage = "Total Degree must be between 50 and 100")]
    public int Degree { get; set; }

    [Required(ErrorMessage = "Minimum passing degree is required")]
    [Range(20, 60, ErrorMessage = "Min Degree must be between 20 and 60")]
    public int MinDegree { get; set; }

    [Range(1, 120, ErrorMessage = "Hours must be between 1 and 120")]
    public int Hours { get; set; }
}
```

### 2. التحقق من جانب الخادم (Server-Side Validation في SaveAdd & SaveEdit):

```
[HttpPost]
public IActionResult SaveAdd(Course course)
{
    // فحص منطقي مخصص: درجة النجاح يجب أن تكون أقل من الدرجة الكلية
    if (course.MinDegree ⩾ course.Degree)
    {
        ModelState.AddModelError("MinDegree", "Min Degree must be less than Total Degree");
    }

    if (ModelState.IsValid)
    {
        courseBL.Add(course);
        return RedirectToAction(nameof(Index));
    }

    ViewData["DeptList"] = new SelectList(departmentBL.GetAll(), "Id", "Name");
    return View("Add", course);
}
```

### 3. التحقق من جانب العميل (Client-Side Validation في Add.cshtml & Edit.cshtml):

يتم عبر استخدام Tag Helpers للربط، وتضمين مكتبات التحقق `ValidationScriptsPartial_` لتعمل في المتصفح فوراً دون Postback:

```
<div asp-validation-summary="All" class="text-danger"></div>

<div class="mb-3">
    <label asp-for="Name" class="form-label"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>
</div>

@section scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

## 3 حل وشرح السؤال الثالث: صفحة نتيجة الطالب في الكورس (شرط: No Conditions in View!)

> 🎯 **اللغز والسر الهندسي للحل:**
> طلب التكليف نصاً: **"colored with green if passed and red if failed .. no conditions in view"**.
> الحل الصحيح لمعمارية MVC هو نقل منطق فحص النجاح وتحديد اللون إلى الـ **Controller Action** وتمريرها جاهزة داخل الـ **ViewModel**، وبذلك تبقى الـ View نظيفة تماماً ولا تحتوي على أي شرط `if@` !

### 1. الـ ViewModel المخصص (StudentCourseResultViewModel):

```
public class StudentCourseResultViewModel
{
    public int StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public int CourseId { get; set; }
    public string CourseName { get; set; } = string.Empty;
    public int Degree { get; set; }
    public int MinDegree { get; set; }

    // قيم جاهزة للعرض في الـ View بدون أي شرط if:
    public string Color { get; set; } = "green";        // "green" أو "red"
    public string Status { get; set; } = "Passed";       // "Passed" أو "Failed"
    public string CssClass { get; set; } = "text-success"; // فئة مساعدة
}
```

### 2. الـ Action داخل CourseController (تستقبل id الطالب و id الكورس):

```
// GET: /Course/Result?studentId=1&courseId=2
public IActionResult Result(int studentId, int courseId)
{
    var student = studentBL.GetById(studentId);
    var course = courseBL.GetById(courseId);
    int studentGrade = studentBL.GetStudentCourse(studentId, courseId)?.Grade ?? 75;

    // الشرط يُحسب هنا في المتحكم وليس في الـ View:
    bool isPassed = studentGrade >= course.MinDegree;

    var viewModel = new StudentCourseResultViewModel
    {
        StudentId = student.Id,
        StudentName = student.Name,
        CourseId = course.Id,
        CourseName = course.Name,
        Degree = studentGrade,
        MinDegree = course.MinDegree,
        Color = isPassed ? "green" : "red",
        Status = isPassed ? "Passed" : "Failed",
        CssClass = isPassed ? "text-success" : "text-danger"
    };

    return View("Result", viewModel);
}
```

### 3. الـ View بدون أي شروط نهائياً (Result.cshtml):

```
@model StudentCourseResultViewModel

<div class="card p-4">
    <h3>نتيجة الطالب: @Model.StudentName</h3>
    <h4>اسم الكورس: @Model.CourseName</h4>
    <p>درجة النجاح الدنيا: @Model.MinDegree</p>

    <!-- لا @if بدون أي شرط ViewModel تلوين الدرجة والحالة مباشرة من الـ -->
    <p>الدرجة: <span style="color: @Model.Color; font-weight: bold;">@Model.Degree</span></p>
    <p>الحالة: <span style="color: @Model.Color; font-weight: bold;">@Model.Status</span></p>
</div>
```

## Bonus أسئلة تكميلية إضافية (من ملف png.01): Architecture & LINQ Execution

### 1- Why we need Architecture in any project? (لماذا نحتاج معمارية برمجية؟)

- **فصل المسؤوليات (Separation of Concerns):** عزل واجهة المستخدم عن قواعد الأعمال وعن التعامل مع قاعدة البيانات.
- **سهولة الصيانة والتطوير (Maintainability):** إجراء أي تعديل مستقبلي في طبقة البيانات دون التأثير على الواجهات.
- **قابلية الاختبار (Testability):** كتابة اختبارات آلية (Unit Tests) للمنطق البرمجي بمعزل عن قاعدة البيانات.
- **تنظيم العمل الجماعي (Scalability & Teamwork):** عمل أعضاء الفريق على طبقات مختلفة في نفس التوقيت دون تعارض.

### 2- What is N-tier Architecture? (ما هي معمارية الطبقات المتعددة؟)

تقسيم النظام إلى طبقات (Tiers/Layers) متعاقبة، حيث تعتمد كل طبقة فقط على الطبقة التي تليها:

- **Presentation Layer (UI):** واجهات المستخدم والـ Controllers.
- **Business Logic Layer (BLL):** قواعد وقوانين المنظومة وحساباتها.
- **Data Access Layer (DAL):** وسيط الاتصال بقاعدة البيانات (DbContext & Repositories).
- **Database Layer:** خادم قاعدة البيانات الفعلي (SQL Server).

### 3- What is Onion Architecture? (ما هي معمارية البصلة؟)

معمارية دائرية حديثة تعتمد على **مبدأ عكس الاعتمادية (Dependency Inversion Principle)**:

- **قلب البصلة (Domain Core):** الكائنات الأساسية (Entities) بدون أي اعتماديات على أي مكتبات خارجية.
- **الطبقة الوسطى:** الـ Domain Services وواجهات التطبيق (Interfaces).
- **الطبقات الخارجية (Outer Rim):** الـ UI وقواعد البيانات والخدمات الخارجية.
- **القاعدة الذهبية:** كل الاعتماديات تتجه **للداخل**، فالنواة لا ترتبط بأي تفاصيل تقنية خارجية.

### 4- Interview Question: Is LINQ slow in execution? (هل LINQ بطيء؟)

**الإجابة الذكية:** الـ LINQ ليس بطيئاً في حد ذاته، بل يترجم لكود فائق السرعة. المشاكل تنتج فقط من **سوء الاستخدام** وعدم فهم الـ **Deferred vs Eager Execution**:

- **التنفيذ المؤجل (Deferred Execution):** مثل `Where` و `Select` . الاستعلام لا يُنفذ لحظة كتابته، بل عند قراءته (مثل دوران `foreach` ).
  *الخطأ الشائع:* عمل دوران `foreach` متعدد على نفس الاستعلام يجعله يذهب للـ Database عدة مرات بلا داعٍ!
- **التنفيذ الفوري (Eager Execution):** مثل `ToList()` و `Count()` و `First()` . يُنفذ فوراً ويجلب النتائج للذاكرة.
  *الخطأ الشائع:* استدعاء `ToList()` قبل الفلترة، مما يسحب آلاف السجلات من الـ DB إلى الـ RAM بدلاً من فلترتها داخل SQL Server!

---

✨ تم الانتهاء بنجاح من إعداد الواجب بالكامل — كود مبني ومُختبر وأسئلة مقالية مبسطة ✨